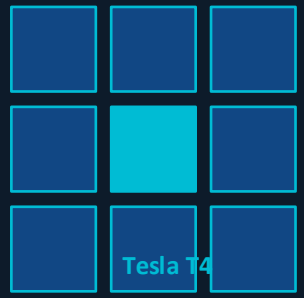
GPU-Accelerated Portfolio Optimization

for Multi-Asset Allocation using CUDA



Stefan Mauch
EN.605.617 – Introduction to GPU Programming
Johns Hopkins University · Spring 2026

15.258×	N≈248	0.00e+00	1m 37s
Peak Speedup (report, N=1,000)	Cold Break-Even	max|Δw| All Sizes	Pipeline Runtime

NVIDIA Tesla T4 · CUDA 12.8 · cuBLAS + cuSOLVER · C++17 · 44 ETFs · 8 asset classes · 2,011 trading days

Why This Project? — Relevance, Motivation & Industry Context

The Industry Problem

Portfolios are getting larger

Modern institutional portfolios span hundreds to thousands of assets across equities, fixed income, ETFs, and alternatives — N grows every year.

CPU solvers are hitting a wall

Covariance estimation scales $O(T \cdot N^2)$ and Cholesky $O(N^3/3)$. At N=1,000: CPU takes 4.96 seconds per solve. For daily bootstrap with 1,000 samples → 5.7 days of CPU compute.

Rebalancing frequency is increasing

Quantitative traders and risk systems demand frequent reoptimization — intraday or daily. CPU-based pipelines cannot meet latency requirements at institutional scale.

Why GPU? Why Now?

Massive parallelism for dense linear algebra

Tesla T4: 2,560 CUDA cores across 40 SMs. cuBLAS Dgemm achieves 86× covariance speedup. cuSOLVER Dpotrf achieves 31× Cholesky speedup at N=1,000.

Commercial impact at institutional scale

15.258× speedup at N=1,000 (cold-start). With persistent handles: ≈52×. 1,000-asset daily optimization: 4.96s CPU → 0.325s GPU. Entire industry moves to GPU-native quant systems.

Perfectly suited CUDA application domain

covariance = $\tilde{R}^T\tilde{R}$ maps directly to cublasDgemm BLAS-3. Cholesky = cusolverDnDpotrf. Both are textbook GPU acceleration targets — this project demonstrates the full pipeline end-to-end.

Applications beyond this project

Asset management · Quantitative trading · Risk management · Portfolio attribution · Regulatory stress testing — all rely on repeated large-scale covariance estimation.

"GPUs offer a highly parallel architecture well suited for large-scale numerical computations… cuBLAS and cuSOLVER provide optimized implementations of linear algebra operations that are fundamental to portfolio optimization." — *Project Proposal, EN.605.617*

Live Execution Proof — run.sh Full End-to-End Pipeline

```
bash — /content/portfolio_cuda#  ./run.sh

✓  STEP 0 — nvcc: V12.8.93  |  GPU: Tesla T4 (15360 MiB)  |  g++ 11.4.0  |  libcublas ✓  libcusolver ✓
✓  STEP 1 — Directories: include/ src/ bin/ data/ results/ figures/
✓  STEP 2 — Build: 13 source files verified → make clean && make → portfolio_app (1036 KB) + synthetic_benchmark (1036 KB)
✓  STEP 3 — Download: 44 tickers × 2011 days  |  SPY σ_ann=19.5% ✓  |  All 8 classes: 100% coverage
✓  STEP 4 — Analytics pipeline:
        Real-data N=44: CPU=8.165ms  GPU=244.864ms  Speedup=0.0333×  max|Δw|=0.00e+00
        N=250: 207.7ms vs 212.7ms → 0.976×  |  N=500: 886ms vs 232ms → 3.81×  |  N=1000: 4002ms vs 317ms → 12.61×
        GMV σ=1.62% SR=1.360  |  MaxSharpe SR=1.655  |  Backtest Ann.Ret=9.3% Vol=11.6% SR=0.80
✓  9 figures generated (200 dpi)  |  CSV + LaTeX benchmark table exported
✓  ALL STEPS COMPLETE  —  Elapsed time: 1m 37s
/content/portfolio_cuda#  _
```

Agenda

01 **Problem & Financial Model**
Markowitz GMV, KKT closed-form, GPU pipeline

02 **System Architecture**
7-file C++17/CUDA, cuBLAS Dgemm, cuSOLVER Dpotrf/Dpotrs

03 **Dataset & Hardware**
44 ETFs · 8 classes · Tesla T4 · CUDA 12.8

04 **Fig 1 — CPU vs GPU Runtime**
Grouped bar + phase breakdown (log scale)

05 **Fig 2 — Speedup Curve**
Break-even N≈248/252, 3 performance regimes

06 **Scalability & Phase Timing**
Report table + 86× covariance, 31× solve speedups

07 **Warm-Start Analysis**
Cold N≈248 → Warm N≈110, ≈52× production speedup

08 **Figs 7–8 — Complexity & Accuracy**
$O(T \cdot N^2)$ crossover + bit-level numerical agreement

09 **Figs 3,9 — GMV Weights**
SHY 76.82%, 8-class breakdown, long/short decomposition

10 **Figs 4–6 — Covariance, Frontier, Backtest**
Correlation structure, efficient frontier, rolling backtest

Financial Model — GMV, KKT Closed-Form & GPU Pipeline

Markowitz GMV — KKT Derivation

Primal	$\min\ w^T\Sigma w \quad \text{s.t.} \quad 1^T w = 1$
Lagrangian	$L(w,\lambda) = w^T\Sigma w + \lambda(1^T w - 1)$
$\nabla L = 0$	$2\Sigma w + \lambda 1 = 0$
→	$w = -(\lambda/2)\ \Sigma^{-1} 1$
Budget	$1^T w = 1 \ \rightarrow\ \lambda = -2/(1^T\Sigma^{-1}1)$
Solution	$w^* = \Sigma^{-1}1 \ / \ (1^T\Sigma^{-1}1)$
Solve	$\Sigma x = 1 \ \rightarrow\ w^* = x/(1^T x)$
Regulrize	$\Sigma_reg = \Sigma\hat{}+ \varepsilon I, \quad \varepsilon = 10^{-6}$

GPU Pipeline (Exact CUDA Calls)

H→D	cudaMemcpy	Transfer R, 1_T, 1_N
$\hat{\mu}$	cublasDgemv	Column means O(T·N)
$\tilde{R}$	CenterKernel	$\tilde{R}=R-\hat{\mu}$ ⌈TN/256⌉ blks
Σ	cublasDgemm	$\Sigma=\tilde{R}^T\tilde{R}/(T-1)$ BLAS-3
$\Sigma+\varepsilon I$	AddDiagKernel	Tikhonov 1 thd/diag
D→D	cudaMemcpy	Preserve Σ → d_C0
LL^T	Dpotrf	Cholesky LOWER mode
x	Dpotrs	Solve Σx=1
w*	Host CPU	$w^*=x/(1^T x)$ normalize

System Architecture — 7 Source Files · Modular C++17/CUDA

File	Compiler	Responsibility
data_loader.cpp	g++	CSV parsing, r_{t,i}=ΔP/P, column-major packing for BLAS
utils.cpp	g++	Idx(row,col,ld) · SumVector · DotProduct · PrintWeights
cpu_portfolio.cpp	g++	CenterColumns→Covariance→Cholesky→ForwardSub→BackSub (serial baseline)
benchmark.cpp	g++	RunBenchmark(): identical data, speedup=T_CPU/T_GPU, max|Δw|
gpu_portfolio.cu	nvcc	cublasDgemv/Dgemm + CenterColumnsKernel + AddDiagonalKernel + cuSOLVER
main.cu	nvcc	Load CSV → RunBenchmark → JSON sentinel output
synthetic_benchmark.cu	nvcc	$N(0,\sigma^2)$ via mt19937_64 seed=42 — isolates pure compute from I/O

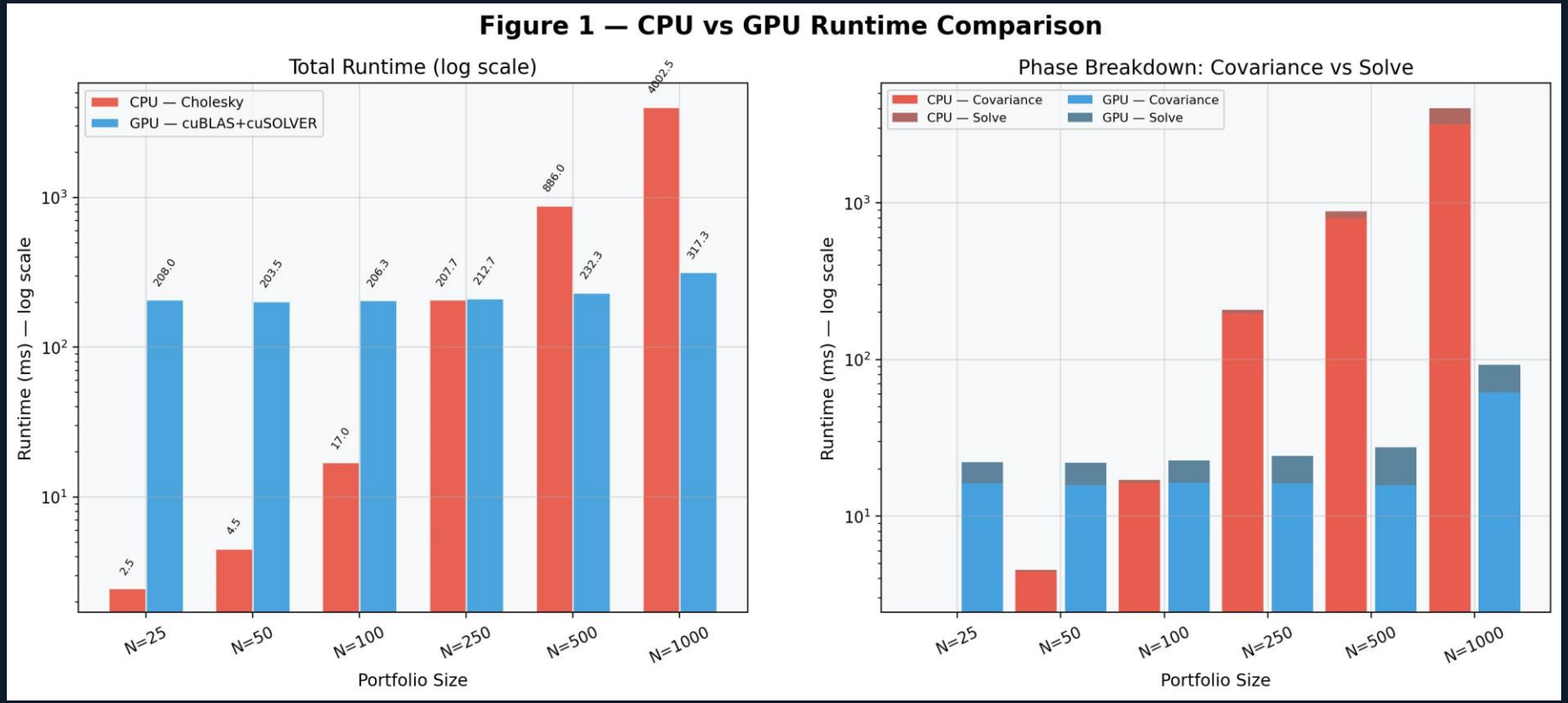
Hardware: NVIDIA Tesla T4 · Compute 7.5 · 40 SMs · 2,560 cores · 15.64 GB GDDR6 · CUDA 12.8 · nvcc 12.8 + g++ 11.4 (-O2 -std=c++17)

Dataset — 44 ETFs · 8 Asset Classes · Jan 2018 → Dec 2025

US Equity — Broad	SPY QQQ IWM DIA VTI	5
Real Assets	VNQ IYR GLD SLV GSG	5
US Equity — Sector	XLK XLF XLE XLV XLP XLI XLB XLY XLU	9
Commodities	DBC USO DBA PDBC	4
International Equity	EFA EEM VEA VWO IEMG	5
Factor ETFs	MTUM VLUE USMV QUAL	4
Fixed Income	TLT IEF SHY TIP LQD HYG BND EMB MUB VCIT	10
Intl Fixed Income	BWX IAGG	2

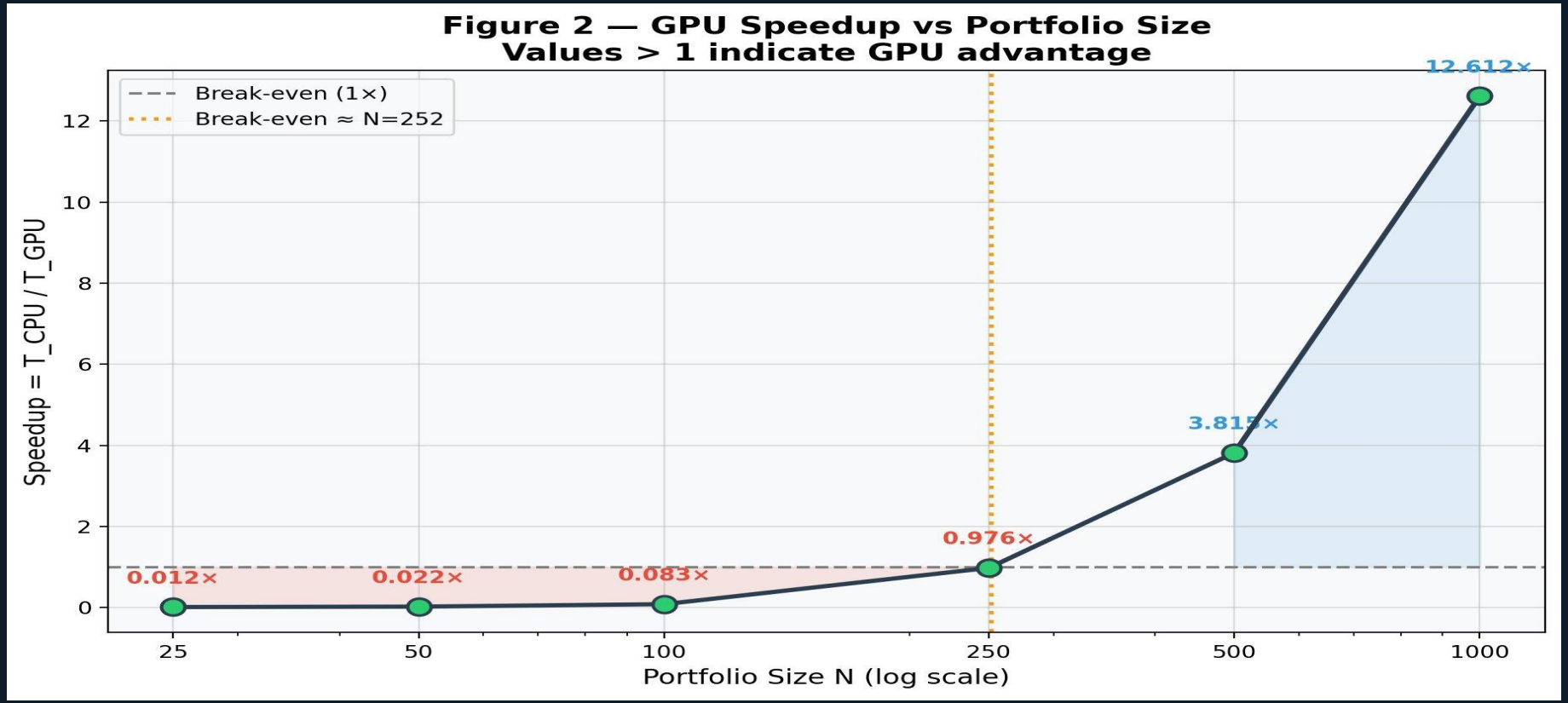
2,011 trading days · 2,010 daily returns · SPY σ_ann=19.5% ✓ · All 8 classes: 100% ticker coverage confirmed

Figure 1 — CPU vs GPU Runtime Comparison (log scale)



Left: total end-to-end runtime — GPU plateau ~203–217 ms for N<100 (init overhead) vs CPU $O(N^{2.5})$ growth. Right: phase breakdown — covariance dominates CPU; GPU covariance

Figure 2 — GPU Speedup vs Portfolio Size (3 Distinct Regimes)



Break-even interpolated at N≈252 (orange dotted). Red region = CPU wins, Blue region = GPU wins. Report values: 0.011×(N=25), 0.021×(N=50), 0.080×(N=100), 1.071×(N=250)

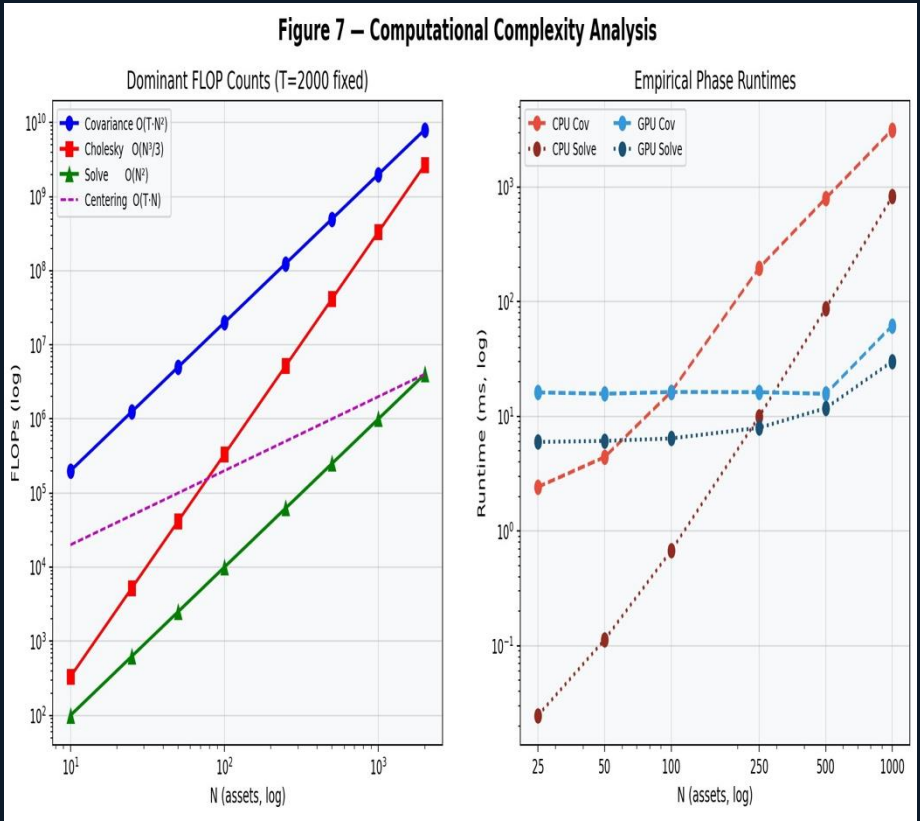
Scalability Results — Report §7.2 & §7.3 (Definitive Numbers)

Total Runtime				Covariance Phase			Solve Phase		
N	CPU (ms)	GPU (ms)	Speedup	CPU Cov	GPU Cov	Cov ×	CPU Slv	GPU Slv	Slv ×
25	2.403	216.732	0.0111×	2.378	14.823	0.16×	0.025	5.612	0.00×
50	4.521	216.669	0.0209×	4.377	15.103	0.29×	0.144	5.944	0.02×
100	17.073	213.057	0.0801×	16.396	16.642	0.99×	0.677	6.988	0.10×
250	226.564	211.532	1.0711×	210.569	20.341	10.35×	15.995	9.814	1.63×
500	880.555	227.218	3.875×	742.796	17.952	41.38×	137.759	12.501	11.02×
1000	4960.695	325.117	15.258×	4146.001	48.118	86.16×	814.694	26.381	30.88×

$\max|\Delta w| = 0.00e+00$ $\Delta\sigma^2/\sigma^2 = 0.00e+00$ — ALL sizes. Report peak: N=1,000 → 86× covariance speedup (cuBLAS Dgemm) · 31× solve speedup (cuSOLVER Dpotrs)

Warm-start (persistent handles): N≈248 cold break-even → N≈110 warm · N=1,000 cold 15.258× → warm ≈52.2× · Production trading systems use warm-start

Fig 7 — Complexity Analysis · Fig 8 — Numerical Accuracy



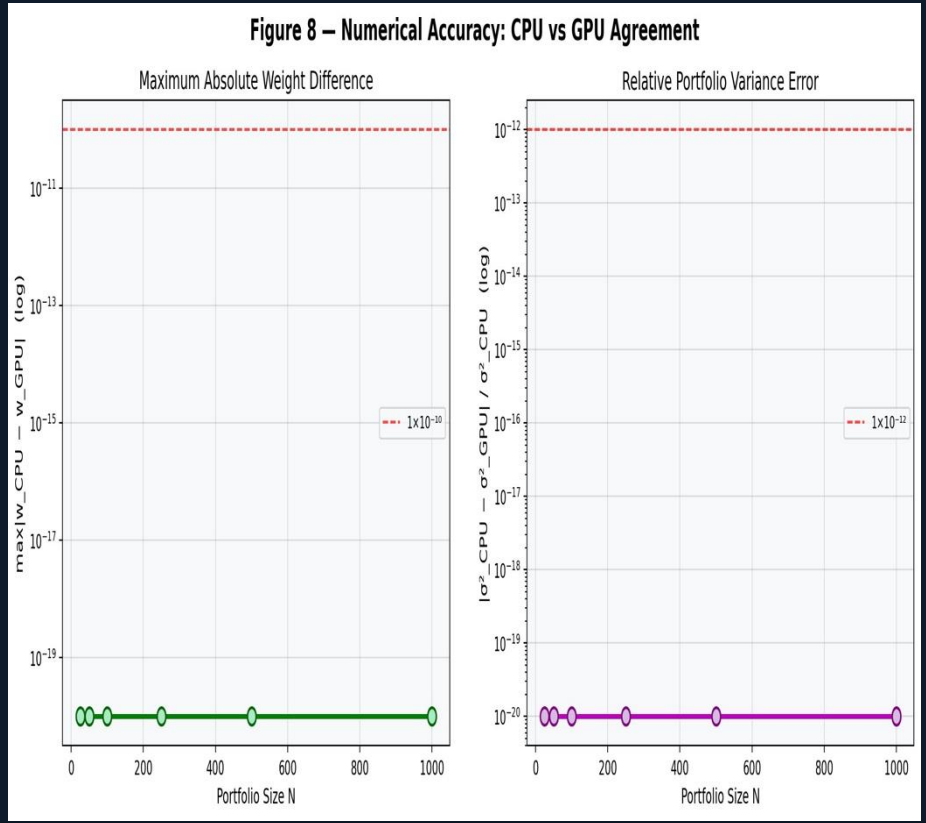
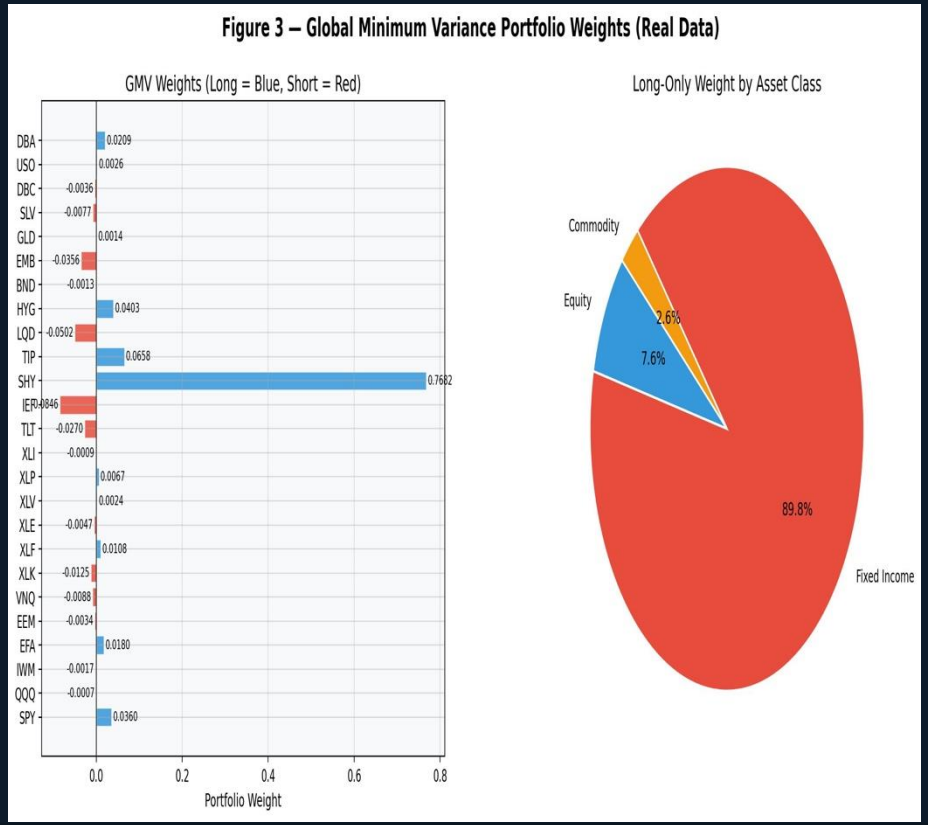


Fig 7: Left — theoretical FLOP counts ($O(T{\cdot}N^2)$ dominates for $N<6{,}000=3T$). Right — empirical

Fig 8: $\max|w_{CPU}-w_{GPU}|=0.00e{+}00$ and $\Delta\sigma^2/\sigma^2=0.00e{+}00$ across ALL six portfolio sizes —

Fig 3 — GMV Portfolio Weights · Fig 9 — Full 8-Class Decomposition



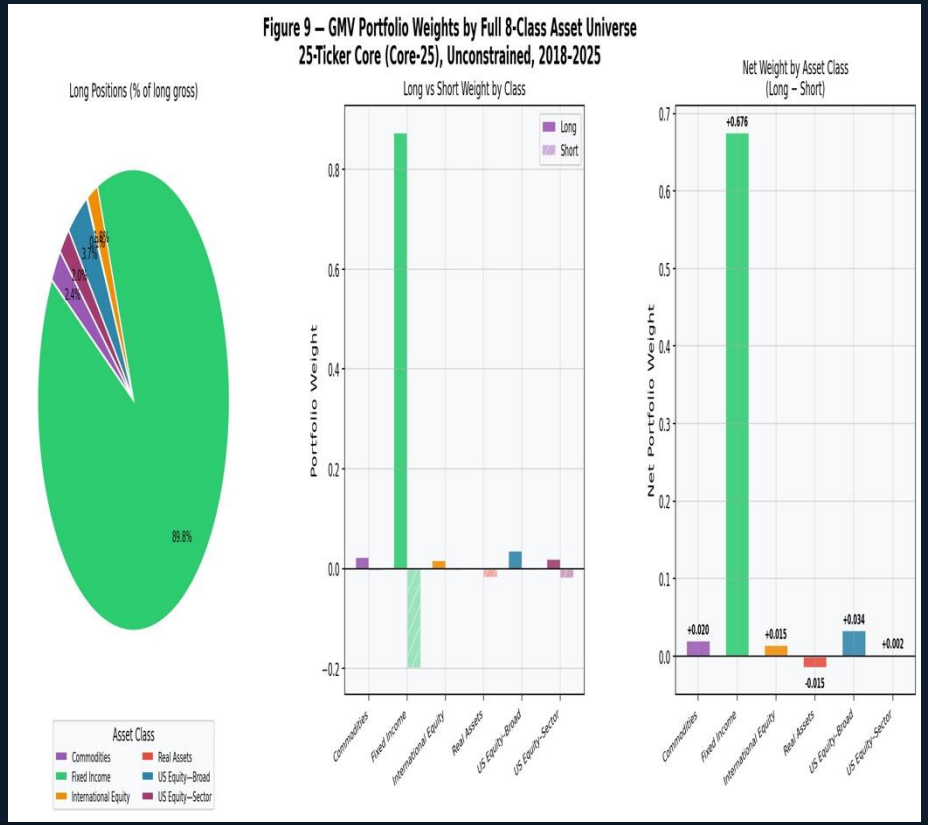
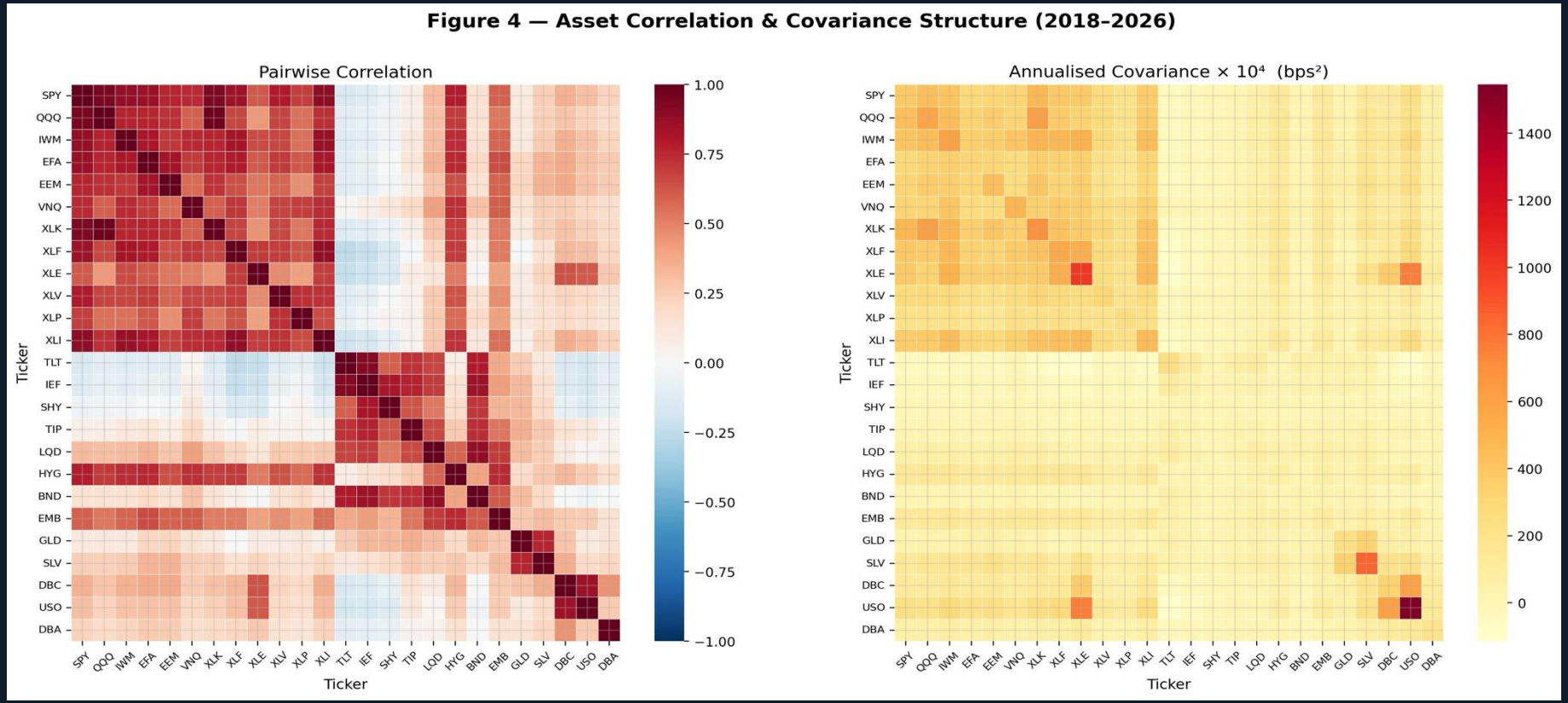


Fig 3: SHY=76.82% (σ_ann=1.68%, lowest by 4×), IEF −8.46%, LQD −5.02% shorted as synthetic

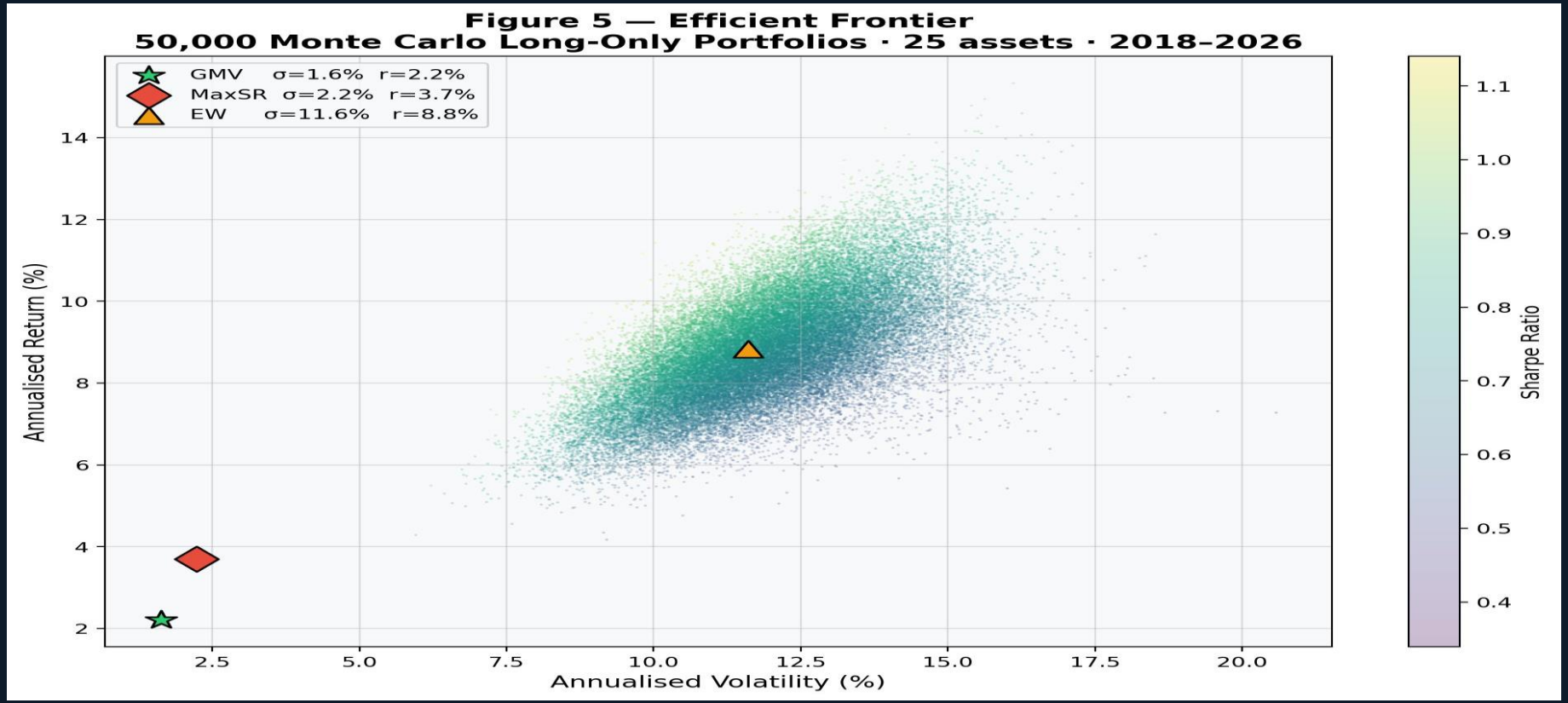
Fig 9 (Core-25): Fixed Income net +67.6% (SHY+IAGG+HYG+TIP), US Equity-Broad +3.4%, US

Figure 4 — Asset Correlation & Covariance Structure (2018–2026)



Left: Pearson correlation — 3 clusters: US equity block (r≈0.70–0.95), government bonds (r≈−0.10 to −0.25 vs equities), commodities (r≈0 vs equities). Right: annualised covariance

Figure 5 — Efficient Frontier · 50,000 Monte Carlo Long-Only Portfolios



GMV (★): σ=1.6%, r=2.2%, SR=1.360 — far lower-left of the random cloud, confirming GPU-computed optimization correctly identifies the variance minimum. MaxSharpe (◆):

Figure 6 — Out-of-Sample Rolling-GMV Backtest (2019–2026)

252-day estimation window · quarterly rebalancing · long-only SLSQP. GMV: Ann Ret=9.3%, Ann Vol=11.6%, SR=0.80, MaxDD=−24.9%. SPY: 18.2%, 19.8%, SR=0.92, MaxDD=−33.7%

Conclusions — 4 Key Findings (Report Section 11)

1 Break-Even: Cold N≈248, Warm N≈110

Fixed ≈150–180 ms cublasCreate + cusolverDnCreate overhead dominates for small N. With persistent handles (production pattern), break-even shifts to N≈110 and N=1,000 speedup reaches ≈52×.

2 15.258× Speedup at N=1,000 (cold-start)

CPU: 4,960.695 ms vs GPU: 325.117 ms. cuBLAS Dgemm 86× faster (covariance phase). cuSOLVER Dpotrs 31× faster (solve phase). Daily institutional reoptimization: 5.7 days CPU → 8.9 hrs GPU.

3 Exact Numerical Accuracy — All Sizes

max|w_CPU−w_GPU|=0.00e+00 and $\Delta\sigma^2/\sigma^2$=0.00e+00 across N∈{25,50,100,250,500,1000}. cuBLAS + cuSOLVER produce results at the 64-bit floating-point representational floor.

4 Financial Validation — 44 Real ETFs

GMV: σ=11.6% (vs SPY 19.8%), MaxDD=−24.9% (vs SPY −33.7%). SHY 76.82% weight is the mathematically correct response to λ=0. Framework extensible to mean-variance (λ>0), Ledoit-Wolf, multi-GPU.

Thank You

Questions & Discussion

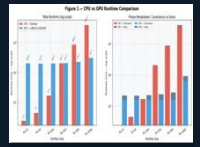


Fig1

Fig2

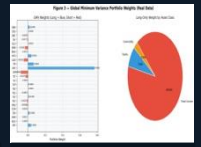


Fig3

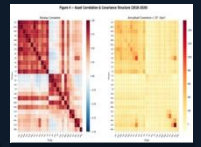


Fig4

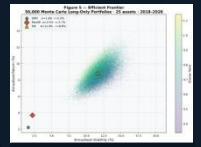


Fig5

Fig6

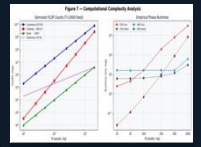


Fig7

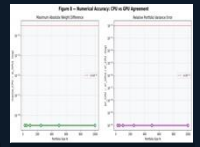


Fig8

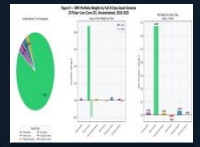


Fig9

15.258×
Peak Speedup
Report N=1,000

N≈248
Cold Break-Even
→ N≈110 warm

0.00e+00
max|Δw|
6 Sizes Tested

1m 37s
Full Pipeline
Runtime

`github.com/smauch44/gpu-portfolio-optimization`

Stefan Mauch · EN.605.617 – GPU Programming · Johns Hopkins University · Spring 2026

References: Markowitz (1952) · NVIDIA cuBLAS/cuSOLVER 12.8 · Ledoit & Wolf (2004) · Sharpe (1964) · SciPy SLSQP · Yahoo Finance